**LAPORAN**

**WORKSHOP SISTEM INFORMASI WEB SERVER**

**(TIF330805)**

**MINGGU 10 PERTEMUAN 1**

**SEMESTER III**

**BKPM ACARA 19**

**Nama:**
**SEPTIAN DWI RAMADHAN PUTRA**

**NIM:**
**E41250282**

**Golongan:**
**A**

**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)**
**TEKNIK INFORMATIKA KAB. NGANJUK**
**JURUSAN TEKNOLOGI INFORMASI**
**POLITEKNIK NEGERI JEMBER**
**TAHUN 2026**

# PENDAHULUAN

## 1. Latar Belakang

API yang sudah dibuat pada acara sebelumnya perlu diuji secara teratur dan dilindungi dari pengguna yang tidak dikenal. Pengujian dilakukan dengan Postman, yaitu aplikasi untuk mengirim request ke API dan melihat responsenya. Agar pengujian rapi, request disimpan dalam sebuah Collection dan alamat server disimpan dalam Environment sehingga dapat diganti dengan mudah.

Untuk autentikasi, API membutuhkan data pengguna. Praktikum ini membuat tabel users dan fitur registrasi (register) pada arsitektur MVC. Kata sandi tidak disimpan apa adanya, melainkan diubah menjadi hash agar tetap aman bila database bocor. Fitur login akan dibuat pada acara berikutnya.

## 2. Tujuan

1. Membuat Postman Collection dan Environment dengan variabel base_url.
2. Membuat tabel users dan Model User.
3. Membuat AuthController dengan aksi register() yang menyimpan kata sandi dalam bentuk hash.
4. Mendaftarkan resource register pada Router.
5. Menguji dan mendokumentasikan endpoint register menggunakan Postman.

## 3. Dasar Teori Singkat

**API Testing.** Pengujian API memastikan setiap endpoint bekerja sesuai rancangan, meliputi metode HTTP, parameter, struktur JSON, kode status, dan penanganan kesalahan.

**Postman.** Aplikasi untuk membuat dan mengirim HTTP request tanpa membuat aplikasi client terlebih dahulu. Request terdiri dari method, URL (endpoint), header, dan body. Response terdiri dari kode status, header, isi (body), waktu respons, dan ukuran. Request dapat dikumpulkan dalam Collection, dan variabel seperti base_url disimpan dalam Environment.

**Autentikasi pada REST API.** Basic Authentication mengirim username dan password (di-encode Base64) pada setiap request. API Key adalah token unik yang dikirim lewat header atau parameter. Bearer Token (termasuk JWT) diperoleh setelah login berhasil dan dikirim pada header Authorization di setiap request berikutnya.

**Penyimpanan kata sandi.** Kata sandi tidak boleh disimpan sebagai teks biasa. PHP menyediakan password_hash() untuk membuat hash yang aman dan password_verify() untuk mencocokkan kata sandi saat login.

## 4. Alat dan Bahan

- XAMPP (Apache dengan mod_rewrite dan MySQL)
- Visual Studio Code
- Postman
- Web Browser
- Database acara_7

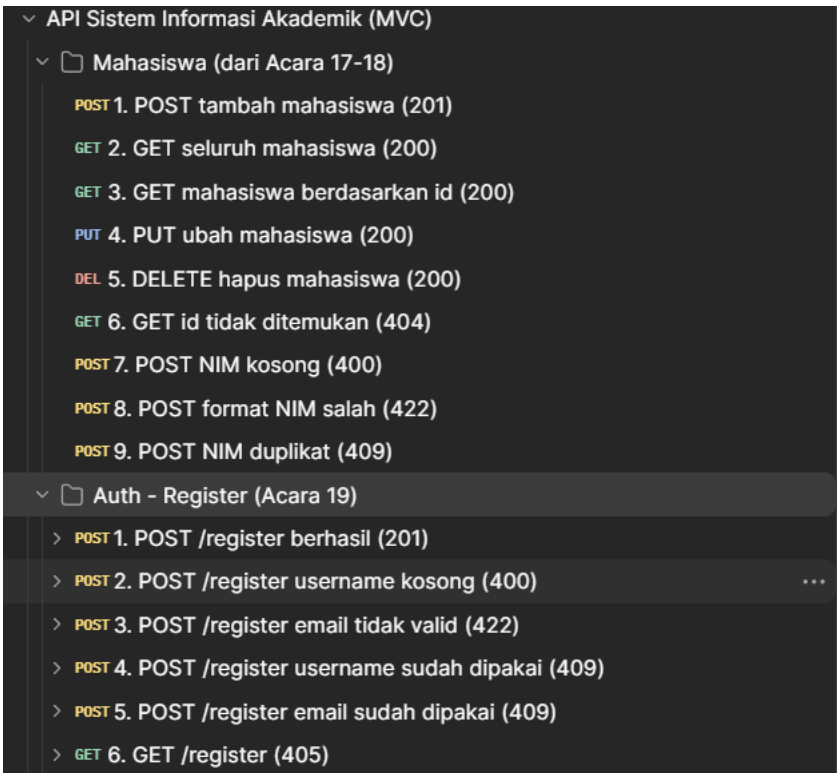
## HASIL PRAKTIK

### 1. Postman Collection dan Environment

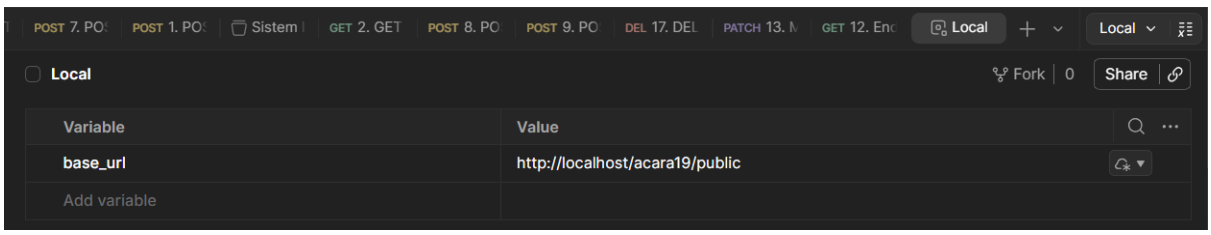
Dibuat Collection bernama "API Sistem Informasi Akademik (MVC)" dan Environment "Local" dengan variabel base_url. Alamat endpoint kini tanpa akhiran .php karena diarahkan oleh Front Controller dan Router. Request mahasiswa dari Acara 17 dan 18 dipindahkan ke Collection ini dengan alamat {{base_url}}/mahasiswa.

| Item | Nilai |
|---|---|
| Collection | API Sistem Informasi Akademik (MVC) |
| Environment | Local |
| Variabel | base_url = http://localhost/sistem-informasi-akademik/public |
| Contoh request | {{base_url}}/mahasiswa dan {{base_url}}/register |

*Tabel 1. Pengaturan Postman*




*Postman Collection*



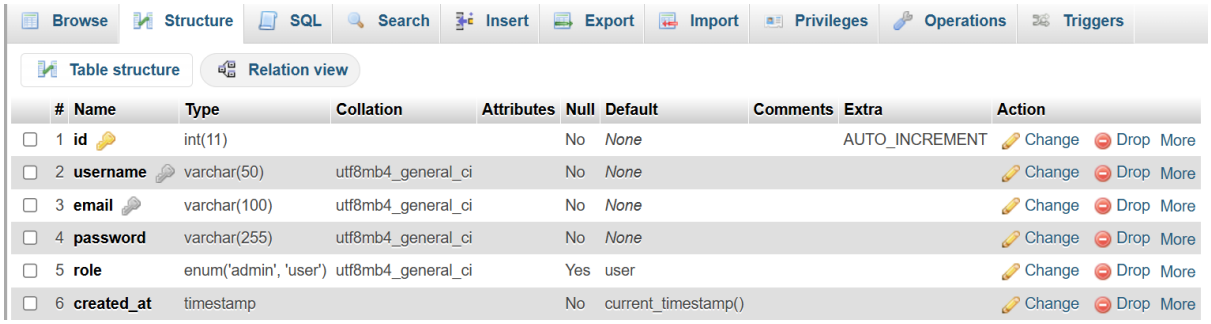

*Environment Local*

## 2. Tabel users

Tabel users ditambahkan pada database acara_7 melalui phpMyAdmin (tab SQL). File acara_7.sql tidak diubah, perintah ini disimpan terpisah pada database/users.sql. Kolom username dan email dibuat UNIQUE agar tidak ada pengguna ganda, kolom password berukuran 255 karakter untuk menampung hash, dan kolom role bernilai awal user.

```sql
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role ENUM('admin','user') DEFAULT 'user',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

*Perintah SQL pembuatan tabel users*




| # | Name | Type | Collation | Attributes | Null | Default | Comments | Extra | Action |
|---|---|---|---|---|---|---|---|---|---|
| 1 | id | int(11) | | | No | None | | AUTO_INCREMENT | Change Drop More |
| 2 | username | varchar(50) | utf8mb4_general_ci | | No | None | | | Change Drop More |
| 3 | email | varchar(100) | utf8mb4_general_ci | | No | None | | | Change Drop More |
| 4 | password | varchar(255) | utf8mb4_general_ci | | No | None | | | Change Drop More |
| 5 | role | enum('admin', 'user') | utf8mb4_general_ci | | Yes | user | | | Change Drop More |
| 6 | created_at | timestamp | | | No | current_timestamp() | | | Change Drop More |

*Tabel users*

## 3. Model User

Model User menyimpan seluruh query tabel users dengan prepared statement: create() untuk menambah pengguna dan findByUsername() untuk mencari berdasarkan username (akan dipakai saat login). Sesuai Tugas Mandiri 1, Model juga memeriksa format email dengan filter_var() dan FILTER_VALIDATE_EMAIL sebelum data disimpan. Username dan email yang sudah dipakai ditolak dengan kode 409.

```php
<?php
// Model User: seluruh query tabel users. Kata sandi yang diterima Model sudah
// berupa hash (dibuat oleh Controller dengan password_hash()).
class User {
    private PDO $db;

    public function __construct() {
        $this->db = Database::getConnection();
    }
```

```php
    public function create(array $data): string {
        $this->validate($data);

        $stmt = $this->db->prepare(
            'INSERT INTO users (username, email, password) VALUES (:u, :e, :p)'
        );
        $stmt->execute([
            ':u' => $data['username'],
            ':e' => $data['email'],
            ':p' => $data['password'],
        ]);
        return $this->db->lastInsertId();
    }

    public function findByUsername(string $username) {
        $stmt = $this->db->prepare('SELECT * FROM users WHERE username = :u');
        $stmt->execute([':u' => $username]);
        return $stmt->fetch();
    }

    // Validasi di Model (Tugas Mandiri 1): format email harus valid
    // (filter_var + FILTER_VALIDATE_EMAIL), serta username dan email tidak
    // boleh sudah terdaftar.
    private function validate(array $data): void {
        if (!filter_var($data['email'], FILTER_VALIDATE_EMAIL)) {
            throw new InvalidArgumentException('Format email tidak valid', 422);
        }
        if (strlen($data['username']) > 50) {
            throw new InvalidArgumentException('Username maksimal 50 karakter',
422);
        }
        if (strlen($data['email']) > 100) {
            throw new InvalidArgumentException('Email maksimal 100 karakter', 422);
        }

        if ($this->findByUsername($data['username'])) {
            throw new InvalidArgumentException('Username sudah terdaftar', 409);
        }

        $stmt = $this->db->prepare('SELECT COUNT(*) FROM users WHERE email = :e');
        $stmt->execute([':e' => $data['email']]);
        if ((int) $stmt->fetchColumn() > 0) {
            throw new InvalidArgumentException('Email sudah terdaftar', 409);
        }
    }
}
```

*app/models/User.php*

**4. AuthController**

AuthController berbeda dari Controller CRUD karena hanya memiliki aksi register(). Aksi ini membaca JSON, memastikan username, email, dan password terisi, lalu mengubah kata sandi menjadi hash dengan password_hash() sebelum diserahkan ke Model. Response hanya berisi id pengguna baru; kata sandi dan hash tidak pernah dikirim balik.

```php
<?php
// AuthController: bukan CRUD standar, hanya aksi register() pada acara ini.
// Aksi login() akan ditambahkan pada acara 20.
class AuthController {
    private User $model;

    public function __construct() {
        $this->model = new User();
    }

    // POST /register
    public function register(): void {
        $data = json_decode(file_get_contents('php://input'), true);

        if (!is_array($data)) {
            Response::json(400, false, 'Body harus berupa JSON yang valid');
        }

        $errors = [];
        foreach (['username', 'email', 'password'] as $field) {
            if (!isset($data[$field]) || !is_scalar($data[$field]) || trim((string)
$data[$field]) === '') {
                $errors[$field] = "Field $field wajib diisi";
            }
        }
        if ($errors) {
            Response::json(400, false, 'Username, email, dan password wajib diisi',
null, $errors);
        }

        // Kata sandi TIDAK disimpan sebagai teks biasa, melainkan hash.
        $user = [
            'username' => trim((string) $data['username']),
            'email'    => trim((string) $data['email']),
            'password' => password_hash((string) $data['password'],
PASSWORD_DEFAULT),
        ];

        try {
            $id = $this->model->create($user);
```

```
        } catch (InvalidArgumentException $e) {
            Response::json($e->getCode() ?: 422, false, $e->getMessage());
        }

        // Response hanya berisi id; password/hash tidak pernah dikirim balik.
        Response::json(201, true, 'Registrasi berhasil', ['id' => $id]);
    }
}
```

*app/controllers/AuthController.php*

### 5. Router

Resource register didaftarkan pada peta Router. Karena register bukan resource CRUD biasa, Router memberi penanganan khusus: hanya method POST yang diterima (selain itu dijawab 405) dan langsung memanggil register(), tanpa melewati pemilihan method CRUD.

```
    private array $map = [
        'mahasiswa' => 'MahasiswaController',
        'prodi'     => 'ProdiController',
        'register'  => 'AuthController',
    ];
```

*Peta resource pada Router*

```
            if ($resource === 'register') {
                if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
                    Response::json(405, false, 'Method tidak diizinkan');
                }
                $controller->register();
                return;
            }
```

*Penanganan khusus resource register*

### 6. Pengujian dan Dokumentasi pada Postman

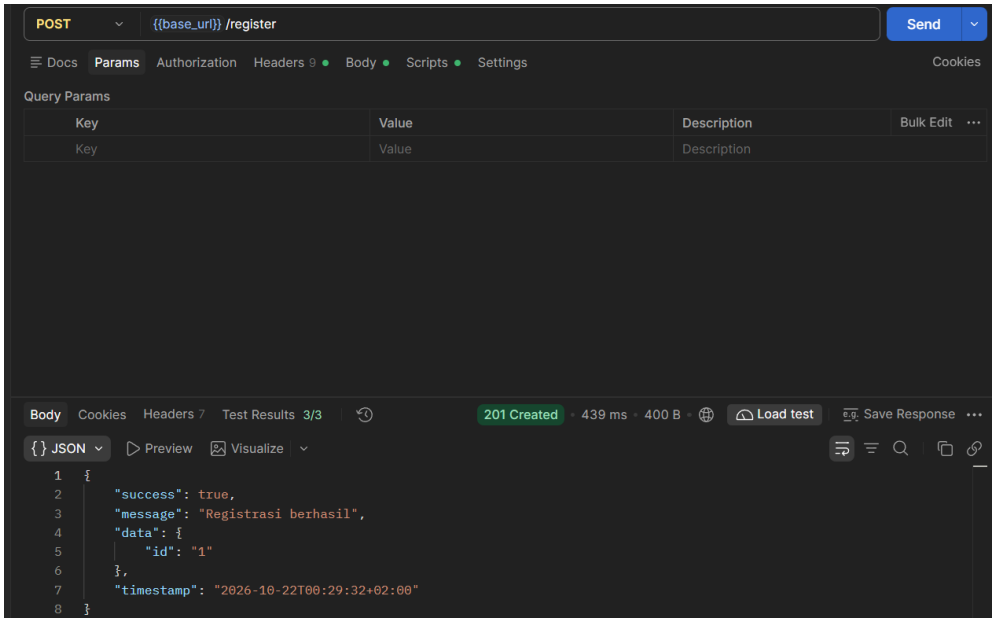
Request POST {{base_url}}/register ditambahkan ke Collection dan diuji dengan beberapa data percobaan. Contoh body request yang benar:

```
{
    "username": "andi123",
    "email": "andi@example.com",
    "password": "rahasia123"
}
```

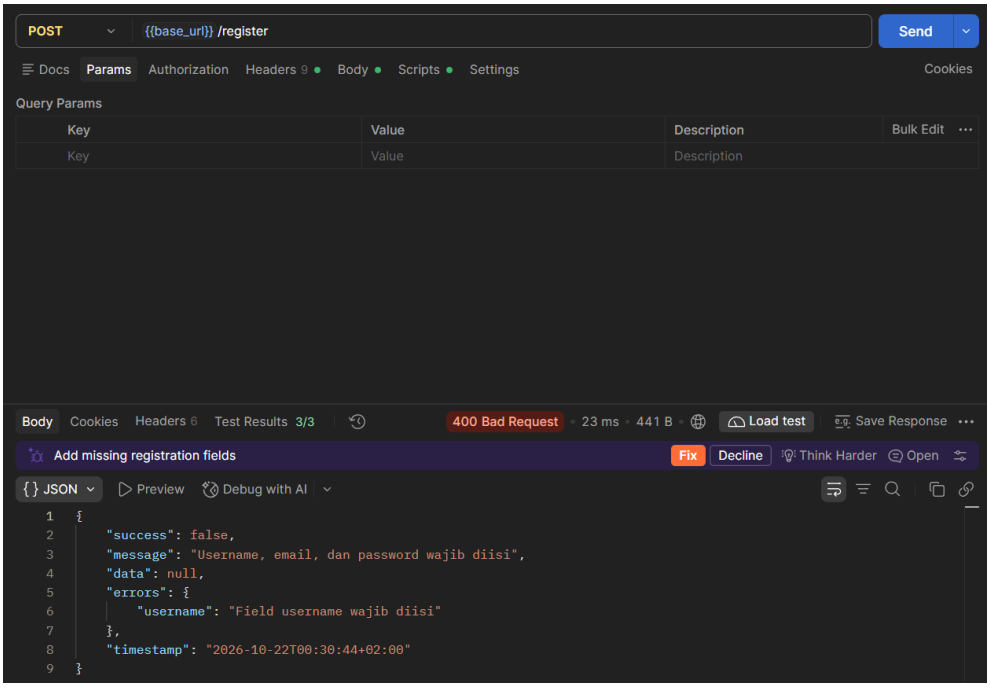
*Contoh body request register*

| No | Request | Status | Hasil yang diharapkan |
|---|---|---|---|
| 1 | POST /register (data valid) | 201 | Registrasi berhasil, response berisi id |
| 2 | POST /register (username kosong) | 400 | Wajib diisi + errors.username |
| 3 | POST /register (email tidak valid) | 422 | Format email tidak valid |
| 4 | POST /register (username sudah dipakai) | 409 | Username sudah terdaftar |
| 5 | POST /register (email sudah dipakai) | 409 | Email sudah terdaftar |
| 6 | GET /register | 405 | Method tidak diizinkan |

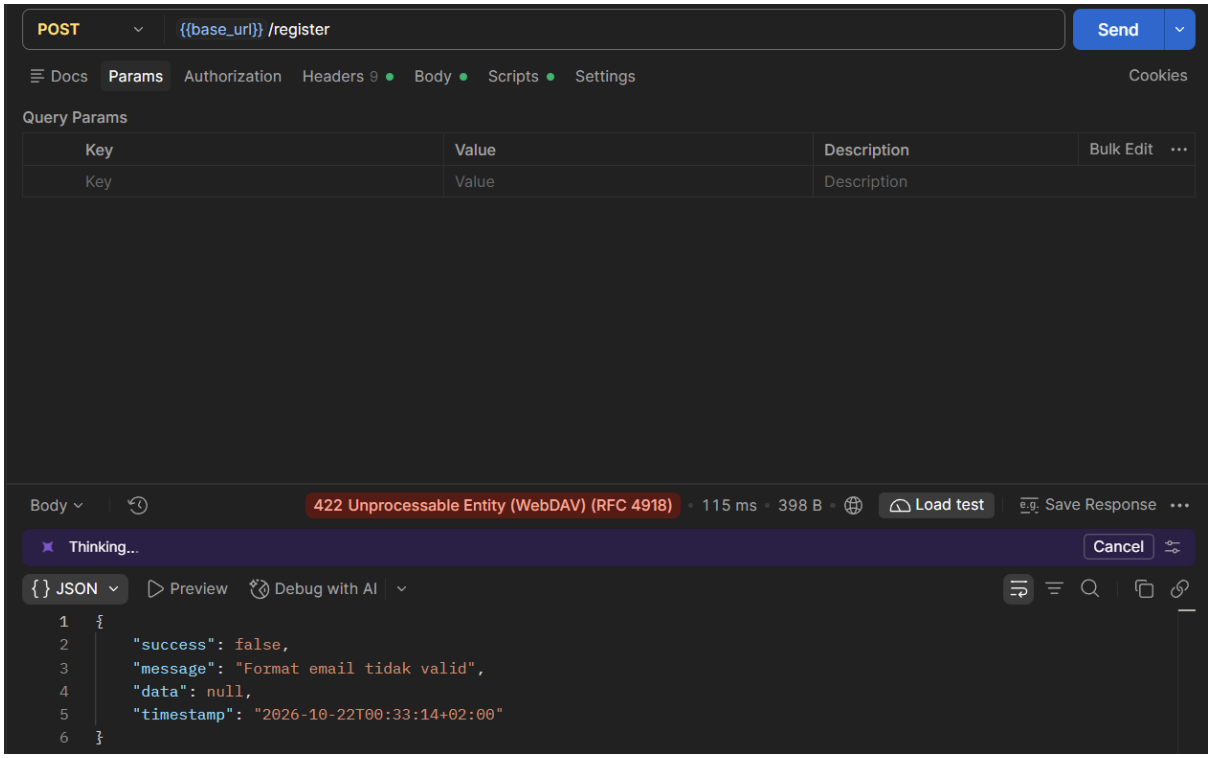
*Tabel Skenario pengujian register*



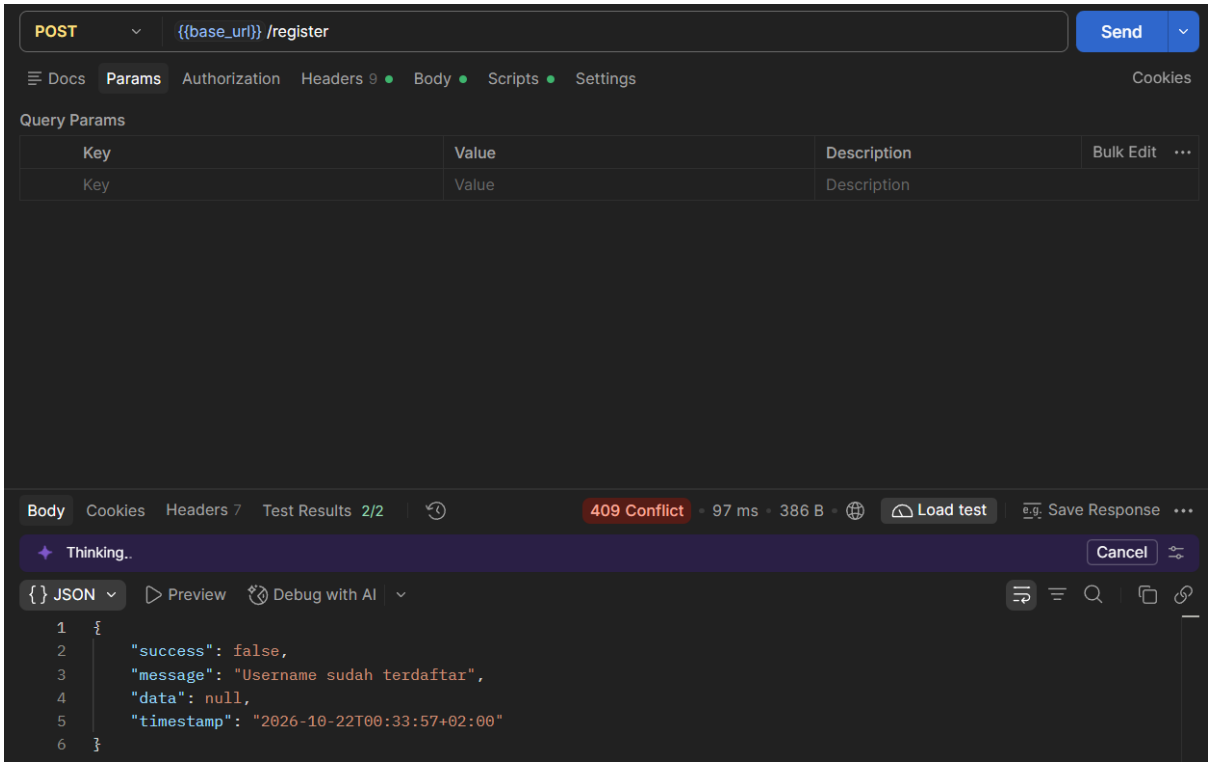

*Registrasi berhasil (201)*



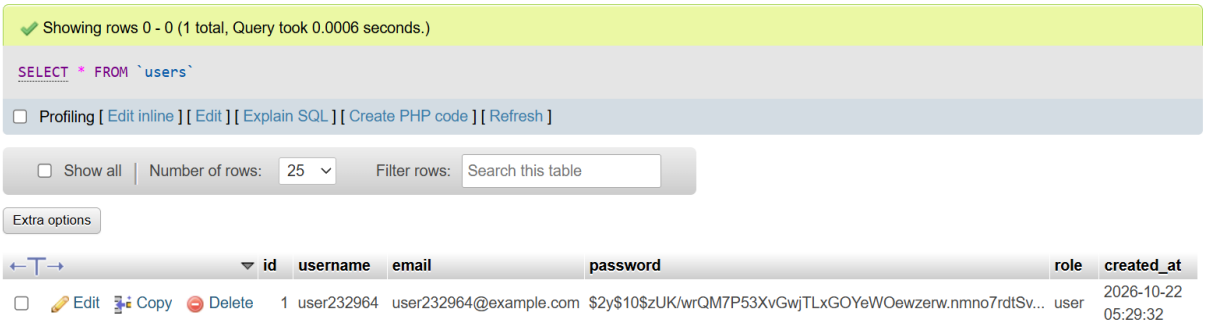

*Data belum lengkap (400)*




*Email tidak valid (422)*




*Username sudah terdaftar (409)*


Showing rows 0 - 0 (1 total, Query took 0.0006 seconds.)

SELECT * FROM `users`

Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

| | id | username | email | password | role | created_at |
|---|---|---|---|---|---|---|
| Edit Copy Delete | 1 | user232964 | user232964@example.com | $2y$10$zUK/wrQM7P53XvGwjTLxGOYeWOewzerw.nmno7rdtSv... | user | 2026-10-22 05:29:32 |

*Kata sandi tersimpan sebagai hash*

Sebagai dokumentasi awal, contoh response sukses dan gagal disimpan pada masing-masing request di Postman (fitur Save as example / Examples). Contoh response:

```
{
    "success": true,
    "message": "Registrasi berhasil",
    "data": {
        "id": "1"
    },
    "timestamp": "2026-10-21T10:30:00+07:00"
}
```

*Contoh response sukses (201)*

```
{
    "success": false,
    "message": "Username, email, dan password wajib diisi",
    "data": null,
    "errors": {
        "username": "Field username wajib diisi"
    },
    "timestamp": "2026-10-21T10:30:00+07:00"
}
```

*Contoh response gagal (400)*

```
{
    "success": false,
    "message": "Format email tidak valid",
    "data": null,
    "timestamp": "2026-10-21T10:30:00+07:00"
}
```

*Contoh response gagal (422)*

```
{
    "success": false,
    "message": "Username sudah terdaftar",
    "data": null,
    "timestamp": "2026-10-21T10:30:00+07:00"
}
```

*Contoh response gagal (409)*

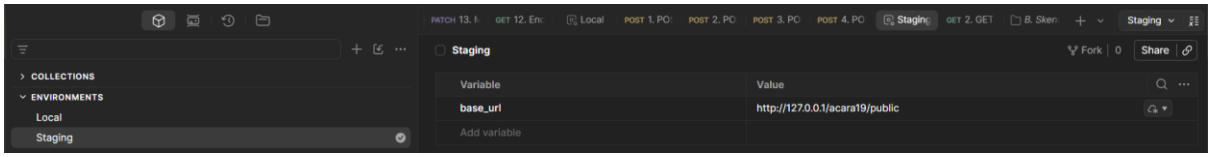
## 7. Tugas Praktik Mandiri

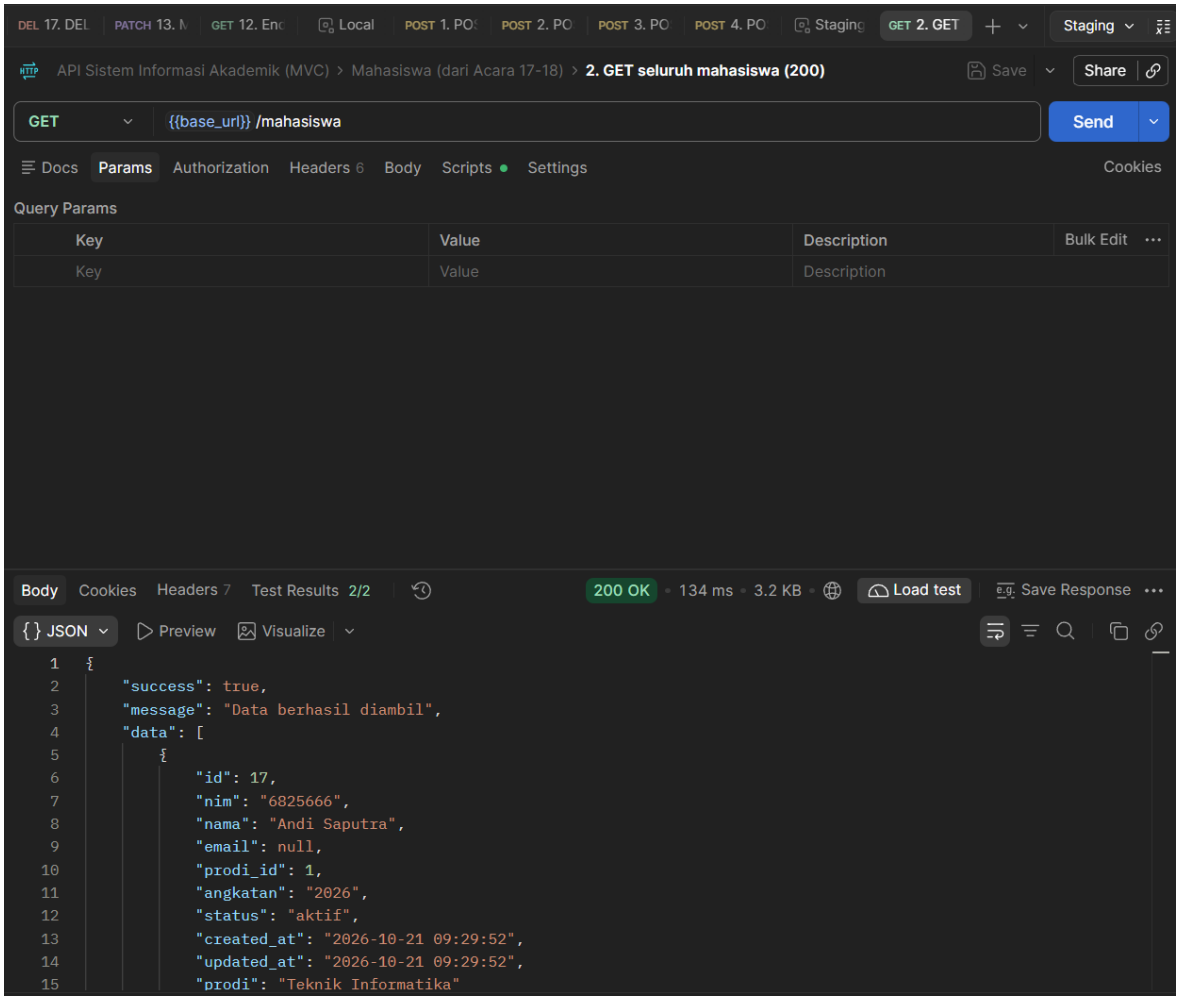
**Tugas 1: validasi email di Model.** Sudah dikerjakan pada Model User (Bagian 3): format email diperiksa dengan filter_var() dan FILTER_VALIDATE_EMAIL. Email yang formatnya salah ditolak sebelum disimpan dan dijawab 422.

**Tugas 2: Environment kedua (staging).** Dibuat Environment "Staging" dengan base_url yang berbeda untuk mensimulasikan lingkungan staging. Request pada Collection tidak perlu diubah, cukup mengganti Environment yang dipilih di pojok kanan atas Postman.

| Environment | base_url |
|---|---|
| Local | http://localhost/acara19/public |
| Staging | http://127.0.0.1/sistem-informasi-akademik/public |

*Tabel Dua Environment pada Postman*




*Environment Staging*

## 8. Pembahasan

- Collection dan Environment membuat pengujian lebih rapi: alamat server cukup diubah di satu tempat (base_url), tidak di setiap request.
- Kata sandi disimpan dalam bentuk hash. Hash dibuat dengan salt acak sehingga dua pengguna dengan kata sandi yang sama memiliki hash yang berbeda, dan kata sandi asli tidak dapat dibaca dari database.
- Pengguna baru selalu berperan sebagai user karena kolom role memakai nilai bawaan database. Role tidak diambil dari request, sehingga pengguna tidak dapat mendaftar sebagai admin.
- Validasi dilakukan di Model dan Controller: kolom kosong dijawab 400, format email salah 422, dan data ganda 409. Detail kesalahan database tidak dikirim ke client.

## KESIMPULAN

1. Postman Collection dan Environment dengan variabel base_url berhasil dibuat, termasuk Environment kedua untuk simulasi staging.
2. Tabel users, Model User, dan AuthController dengan aksi register() berhasil dibuat, dan kata sandi disimpan dalam bentuk hash dengan password_hash().
3. Resource register berhasil didaftarkan pada Router dan hanya menerima method POST.
4. Endpoint register berhasil diuji dan didokumentasikan pada Postman, termasuk validasi email dengan filter_var().

# DAFTAR PUSTAKA

Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Politeknik Negeri Jember.